The value of such a sum can be efficiently calculated using sum formulas. For example, the term z_0 corresponds to the sum $(y-x+1)z_0$, and the term $z_1 u$ corresponds to the sum

$$z_1(0+1+\cdots+y-x) = z_1 \frac{(y-x)(y-x+1)}{2}.$$

When propagating an update in the tree, the indices of $p(u)$ change, because in each range $[x,y]$, the values are calculated for $u = 0,1,\ldots,y-x$. However, this is not a problem, because $p'(u) = p(u+h)$ is a polynomial of equal degree as $p(u)$. For example, if $p(u) = t_2u^2 + t_1u - t_0$, then

$$p'(u) = t_2(u+h)^2 + t_1(u+h) - t_0 = t_2u^2 + (2ht_2 + t_1)u + t_2h^2 + t_1h - t_0.$$

Dynamic trees

An ordinary segment tree is static, which means that each node has a fixed position in the array and the tree requires a fixed amount of memory. In a **dynamic segment tree**, memory is allocated only for nodes that are actually accessed during the algorithm, which can save a large amount of memory.

The nodes of a dynamic tree can be represented as structs:

```
struct node {
    int value;
    int x, y;
    node *left, *right;
    node(int v, int x, int y) : value(v), x(x), y(y) {}
};
```

Here `value` is the value of the node, `[x,y]` is the corresponding range, and `left` and `right` point to the left and right subtree.

After this, nodes can be created as follows:

```
// create new node
node *x = new node(0, 0, 15);
// change value
x->value = 5;
```

Sparse segment trees

A dynamic segment tree is useful when the underlying array is *sparse*, i.e., the range $[0,n-1]$ of allowed indices is large, but most array values are zeros. While an ordinary segment tree uses $O(n)$ memory, a dynamic segment tree only uses $O(k \log n)$ memory, where k is the number of operations performed.

A **sparse segment tree** initially has only one node $[0,n-1]$ whose value is zero, which means that every array value is zero. After updates, new nodes are dynamically added to the tree. For example, if $n = 16$ and the elements in positions 3 and 10 have been modified, the tree contains the following nodes: